Transformer Inference

Worked Problems - Condensed Review

Lucas Yan · JAX Scaling Book, Chapter 7

Central idea. Decode is usually a **data-movement problem**, not a FLOPs problem. Start from tensor shapes, convert to bytes, convert bytes to time, then identify the largest bottleneck.

1. Symbols that must stay separate

B	concurrent decode requests / tokens
T	tokens processed now; decode usually $T = 1$
S	cached context length
L, D, F	layers, model width, MLP width
N, K, H	query heads, KV heads, head dimension
E, k	total experts, activated experts/token
b_w, b_{KV}	bytes per weight / KV element

Do not mix these: N makes queries; K determines cached heads. E experts are stored; only k are used by one token. The 2 in KV memory means *key and value*, not $k = 2$ experts.

2. Dense-model parameter count

For a gated MLP and shared input/output embedding:

$$\boxed{P = 3LDF + 2LDH(N + K) + DV}$$

$$P_{\text{MLP}} = 3LDF, \qquad P_{\text{attn}} = 2LDH(N + K).$$

Attention decomposition:

$$\underbrace{W_Q + W_O}_{2DNH} + \underbrace{W_K + W_V}_{2DKH}.$$

Worked model:

$$P = 12.885\text{B} + 5.369\text{B} + 0.132\text{B} \approx \boxed{18.4\text{B}}.$$

Parameters are counts, not bytes:

$$\boxed{M_{\text{weights}} = b_w P}$$

Int8 uses 1 byte/parameter; bf16 uses 2.

3. KV cache

Only keys and values survive for future tokens:

$$\boxed{m_{\text{KV/token}} = 2LKH\, b_{\text{KV}}}$$

For one sequence:

$$\boxed{M_{\text{KV,seq}} = S(2LKH)b_{\text{KV}}}$$

Conceptual shape:

$$\text{KV}[L, 2, B, S, K, H].$$

Worked model, int8:

$$2(64)(8)(256) = 262{,}144 \text{ bytes/token} \approx \boxed{262 \text{ kB/token}}.$$

At $S = 128{,}000$:

$$M_{\text{KV,seq}} \approx \boxed{33.55 \text{ GB}}.$$

4. Memory-capacity questions

Weights are a fixed cost; every request adds another KV cache:

$$\boxed{B_{\max} = \left\lfloor \frac{N_c M_{\text{HBM}} - M_{\text{weights}}}{M_{\text{KV,seq}}} \right\rfloor}$$

For 16 TPU v5e chips:

$$B_{\max} = \left\lfloor \frac{256 - 18.4}{33.55} \right\rfloor = \boxed{7}.$$

Reducing $K : 8 \to 1$ makes KV memory 8× smaller:

$$\boxed{B_{\max} \approx 56}.$$

5. Bytes become latency

Universal conversion:

$$\boxed{\text{time} = \frac{\text{bytes moved}}{\text{bandwidth}}}$$

Fully sharded weight loading:

$$\boxed{T_{\text{weights}} = \frac{b_w P}{N_c W_{\text{HBM}}}}$$

Worked result:

$$\frac{18.4 \times 10^9}{16(8.2 \times 10^{11})} \approx \boxed{1.4 \text{ ms}}.$$

All chips load their shards simultaneously; use aggregate bandwidth $N_c W_{\text{HBM}}$.

6. Prefill versus decode

	Prefill	**Decode**
Tokens now	many	one/request
Weight reuse	high	low
KV action	create	read history
Typical limit	compute	HBM bandwidth
Extra scaling	sequence	model/KV

Each decode step produces one new token but reads S previous KV positions. Long context therefore makes attention bandwidth-heavy.

7. Decode-step latency

$$T_{\text{KV}} = \frac{BM_{\text{KV,seq}}}{N_c W_{\text{HBM}}}, \qquad T_{\text{math}} = \frac{2BP_{\text{act}}}{N_c C}.$$

Core lower bound:

$$\boxed{T_{\text{step}} \approx T_{\text{KV}} + \max(T_{\text{weights}}, T_{\text{math}})}$$

Why add then max?

- Attention and linear blocks are sequential, so their times add.
- Inside a linear block, compute and weight loading compete; slower wins.

At $B = 7, S = 128{,}000$:

$$T_{\text{KV}} \approx 17.9 \text{ ms}, \quad T_{\text{weights}} \approx 1.4 \text{ ms}, \quad T_{\text{math}} \approx 0.041 \text{ ms}.$$

$$\boxed{T_{\text{step}} \approx 19.3 \text{ ms}}$$

before fixed collective/runtime overheads.

8. Rooflines

A roofline is a **crossing of two candidate bottlenecks**.

- "When FLOPs-bound?" Set $T_{\text{math}} = T_{\text{HBM}}$.
- "How far can tensor parallelism scale?" Set $T_{\text{ICI}} = T_{\text{HBM}}$.

Decode tensor-parallel limit:

$$\boxed{p_{\max} \approx \frac{F}{B\beta}, \qquad \beta = \frac{W_{\text{HBM}}}{W_{\text{ICI}}}}$$

Prefill approximation:

$$\boxed{p_{\text{prefill}} \approx F/2200}$$

For $F = 16{,}384$, $B = 7$, $\beta \approx 8$:

$$p_{\max}^{\text{decode}} \approx 293 \gg 16, \qquad p_{\text{prefill}} \approx 7.4.$$

Interpretation: 16-way decode remains HBM-limited in the simplified model; prefill should use roughly 4–8-way tensor parallelism, then sequence parallelism.

9. KV-cache sharding

Given $\text{KV}[2, B, S, K, H]$, consider:

1. K **first**: different devices own KV heads.
2. B **second**: devices own different requests.
3. S **third**: devices own context segments.
4. Avoid replication when the cache is large.

Head sharding is limited to K ways. Beyond that, batch/sequence sharding adds collectives, commonly AllToAlls for layout changes.

Do not invent communication bytes by adding $BD + DF + BF$. Count only tensors actually transmitted by each collective. Stationary weights remain in HBM; activations usually move over ICI.

MoE and Large-Scale Sharding

10. MoE: stored versus activated

Total stored parameters:

$$\boxed{P_{\text{total}} = 3ELDF + 2LDH(N+K) + DV}$$

Used by one token:

$$\boxed{P_{\text{activated}} = 3kLDF + 2LDH(N+K) + DV}$$

Forward FLOPs for T tokens:

$$\boxed{\text{FLOPs} \approx 2P_{\text{activated}}T}$$

Worked values for $E = 16, k = 2$:

$$P_{\text{total}} \approx \boxed{212\text{B}}, \qquad P_{\text{activated}} \approx \boxed{31.2\text{B}},$$

$$\text{FLOPs} \approx \boxed{62.4 \times 10^9 T}.$$

MoE adds E times the expert storage but only k times expert compute. Therefore:

$$\boxed{B_{\text{crit,MoE}} \approx B_{\text{crit,dense}} \frac{E}{k}}$$

$$240\frac{16}{2} = \boxed{1920 \text{ tokens}}.$$

MoE changes the MLP, not attention. The KV-cache formula $2LKH$ is unchanged.

11. Expert sharding

Expert weight tensor:

$$W[E, D, F] \longrightarrow W[E_Z, D_X, F_Y].$$

- Z: which experts a chip owns.
- Y: which F columns of that expert.
- X: which D rows; training/FSDP dimension here.

At inference, $X = 1$. Idealized weight balance:

$$\boxed{M_{\text{weights/chip}} \approx \frac{b_w P}{YZ}}$$

$$T_{\text{HBM}} \approx \frac{b_w P}{YZ\,W_{\text{HBM}}}, \qquad M_{\text{free}} \approx M_{\text{chip}} - \frac{b_w P}{YZ}.$$

Minimum capacity:

$$\boxed{N_{\min} = \left\lceil \frac{b_w P}{M_{\text{chip}}} \right\rceil}$$

then round up to a supported, divisible topology.

For the 212B MoE in bf16:

- $Y = 8, Z = 16$ (128 chips): $\approx$ 4.0 ms weight load, $\approx$ 12.7 GB free/chip.
- Smallest practical slice: $\boxed{32 \text{ chips}}$.

Memory rule: experts stay; tokens travel. Expert parallelism avoids moving huge weights by routing much smaller activations to expert-owning chips.

12. 2D weight-stationary sharding

Split both D and F so local weight tiles become more balanced. Let total chips be $N = XYZ$.

Two local matmuls:

$$\boxed{T_{\text{math}} = \frac{4BDF}{NC}}$$

Collective time:

$$\boxed{T_{\text{comms}} = \frac{2BD}{XW_{\text{ICI}}} + \frac{4BF}{YZ\,W_{\text{ICI}}}}$$

For $F = 4D$:

$$T_{\text{comms}} = \frac{2BD}{W_{\text{ICI}}}\left(\frac{1}{X} + \frac{8}{YZ}\right).$$

At the optimum, balance the communication terms:

$$\frac{1}{X} = \frac{8}{YZ} \quad \Longrightarrow \quad \boxed{YZ = 8X}.$$

Using $X(YZ) = N$:

$$\boxed{X = \sqrt{\frac{N}{8}}, \qquad YZ = \sqrt{8N}}.$$

Optimal communication:

$$\boxed{T^{2D}_{\text{comms}} \approx \frac{\sqrt{128}\,BD}{\sqrt{N}\,W_{\text{ICI}}}}.$$

It beats the chapter's traditional baseline when:

$$\boxed{N > 72} \qquad \text{or generally} \qquad \boxed{N > 18F/D}.$$

Meaning: 2D sharding adds collectives, but makes the communicated buffers shrink with scale. It pays off only after enough chips.

13. Recurring traps

1. **Parameters vs bytes:** multiply by bytes/parameter.
2. **Total vs activated:** storage uses E; token FLOPs use k.
3. **Query vs KV heads:** N affects query projection; K affects cache size.
4. **Wrong roofline pair:** choose math/HBM or ICI/HBM based on the question.
5. **Wrong aggregation:** fully sharded chips load simultaneously; both bandwidth and compute scale with chip count.
6. **Wrong max/sum:** sequential stages add; competing bottlenecks use the maximum.

14. Seven-step solving checklist

1. Write the tensor shape.
2. Count elements.
3. Multiply by bytes/element.
4. Mark cost as fixed, per token, or per request.
5. Apply sharding dimensions.
6. Convert work/bytes into candidate times.
7. Name the largest bottleneck and sanity-check units.

Mastery test. If K halves, E doubles, and k stays fixed: KV memory halves; total expert parameters roughly double; activated expert parameters stay fixed; the MoE critical batch roughly doubles.

Reference: https://jax-ml.github.io/scaling-book/inference/

Summary derived from Worked Problems 1–7 and the accompanying tutoring discussion.